Connor Petri

Assignment 3

Design Patterns

Controller:

ActionListener (this is from a pop-up window which prompts the user for name and file path):

```
JButton okButton = new JButton( text: "OK");
okButton.addActionListener( ActionEvent e -> {
    if (!nameField.getText().isEmpty() && !filePathField.getText().isEmpty()) {
        controller.addPhoto(nameField.getText(), filePathField.getText());
        parent.repaint();
        dispose();
    }
});
```

Controller Method:

```
public void addPhoto(String name, String filePath) {  1 usage  Connor Petri *
    model.addPhoto(new Photo(name, filePath));
    it = model.iterator();
}
```

Model:

```
private final ArrayList<Photo> photos = new ArrayList<>();  19
private ISortingStrategy sortingStrat = new SortByDate();  7 us

public void addPhoto(Photo p) {  1 usage  new *
    photos.add(p);
    sortingStrat.sort(photos);
}

public boolean deletePhoto(Photo p) {  1 usage  new *
    return photos.remove(p);
}
```

View:

```
@Override  2 usages  Connor Petri
public void paintComponent(Graphics g) {
    super.paintComponent(g);
    Graphics2D g2d = (Graphics2D) g;
    Photo selectedPhoto = controller.getSelectedPhoto();

    int y = 20;
    for (Photo p : controller.getPhotos()) {
        if (p.getPath().equals(selectedPhoto.getPath())) {
            g2d.setColor(Color.BLUE);
            g2d.fillRect( i: 0,  i1: y - 15, getWidth(), ROW_HEIGHT);
            g2d.setColor(Color.WHITE);
        } else {
            g2d.setColor(Color.BLACK);
        }

        g2d.drawImage(p.getThumbnail(), PADDING,  i1: y - 12, THUMB_SIZE, THUMB_SIZE,  imageObserver: this);
        int textX = PADDING + THUMB_SIZE + PADDING;
        int textY = y + (ROW_HEIGHT / 2) - 10;
        g2d.drawString(p.getName(), textX, textY);
        y += ROW_HEIGHT + PADDING;
    }
}
```

Strategy Pattern

```
public class SortByDate implements ISortingStrategy {  2 usages  Connor Petri
    public List<Photo> sort(List<Photo> photos) {  4 usages  Connor Petri
        Collections.sort(photos, ( Photo p1,  Photo p2) -> p2.getDateAdded().compareTo(p1.getDateAdded()));
        return photos;
    }
}
```

```
public class SortBySize implements ISortingStrategy {  1 usage  Connor Petri
    public List<Photo> sort(List<Photo> photos) {  4 usages  Connor Petri
        Collections.sort(photos, ( Photo p1,  Photo p2) -> Long.compare(p1.getSizeBytes(), p2.getSizeBytes()
        return photos;
    }
}
```

```
public class SortByDate implements ISortingStrategy {  2 usages  Connor Petri
    public List<Photo> sort(List<Photo> photos) {  4 usages  Connor Petri
        Collections.sort(photos, ( Photo p1,  Photo p2) -> p2.getDateAdded().compareTo(p1.getDateAdded()));
        return photos;
    }
}
```

```
private ISortingStrategy sortingStrat = new SortByDate();  7 usages

public void addPhoto(Photo p) {  1 usage  new *
    photos.add(p);
    sortingStrat.sort(photos);
}
```

Iterator Pattern

```
public class PhotoAlbumModel {  2 usages  Connor Petri *
    public class Iterator implements IPhotoAlbumIterator {  2 usages  Connor Petri *

        public Iterator() {  1 usage  new *
            if (!photos.isEmpty()) {
                current = photos.getFirst();
            }
        }

        public boolean hasNext() {  2 usages  Connor Petri
            if (photos.isEmpty()) { return false; }

            if (current == null) { return true; }

            int i = photos.indexOf(current);
            return i < photos.size() - 1; // Check if i is larger than the index
        }

        public boolean hasPrevious() {  1 usage  Connor Petri
            if (current == null) { return false; }
            return photos.indexOf(current) > 0;
        }

        public Photo next() {  1 usage  Connor Petri *
            if (!hasNext()) { return null; }

            if (current == null) {
                current = photos.getFirst();
            } else {
                current = photos.get(photos.indexOf(current) + 1);
            }
            return current;
        }

        public Photo previous() {  1 usage  Connor Petri
            current = photos.get(photos.indexOf(current) - 1);
            return current;
        }

        public Photo current() {  no usages  Connor Petri
            return current;
        }
    }
```